Балтийский государственный технический университет
«ВОЕНМЕХ» им. Д. Ф. Устинова

Кафедра И5 «Информационные системы и программная инженерия»

Практическая работа №2
по дисциплине «Информатика: Основы программирования»
на тему «Ветвления и циклы»

Выполнил:
Студент Праудин Д.с
Группа 0717b12

Преподаватель:
Палехова О.А.

Санкт-Петербург
2020 г.

Задача 1. Вычислить значение функции 1. $f(x,y)=\begin{cases} y/x, \text{ если } x\neq 0, \\ y^2, \text{ если } x=0, y<0, \\ 5 \text{ в остальных случаях} \end{cases}$, используя условную операцию.

Исходные данные:

Аргументы функции x и y. Так как значения x и y могут быть любыми, объявим соответствующие переменные типа double.

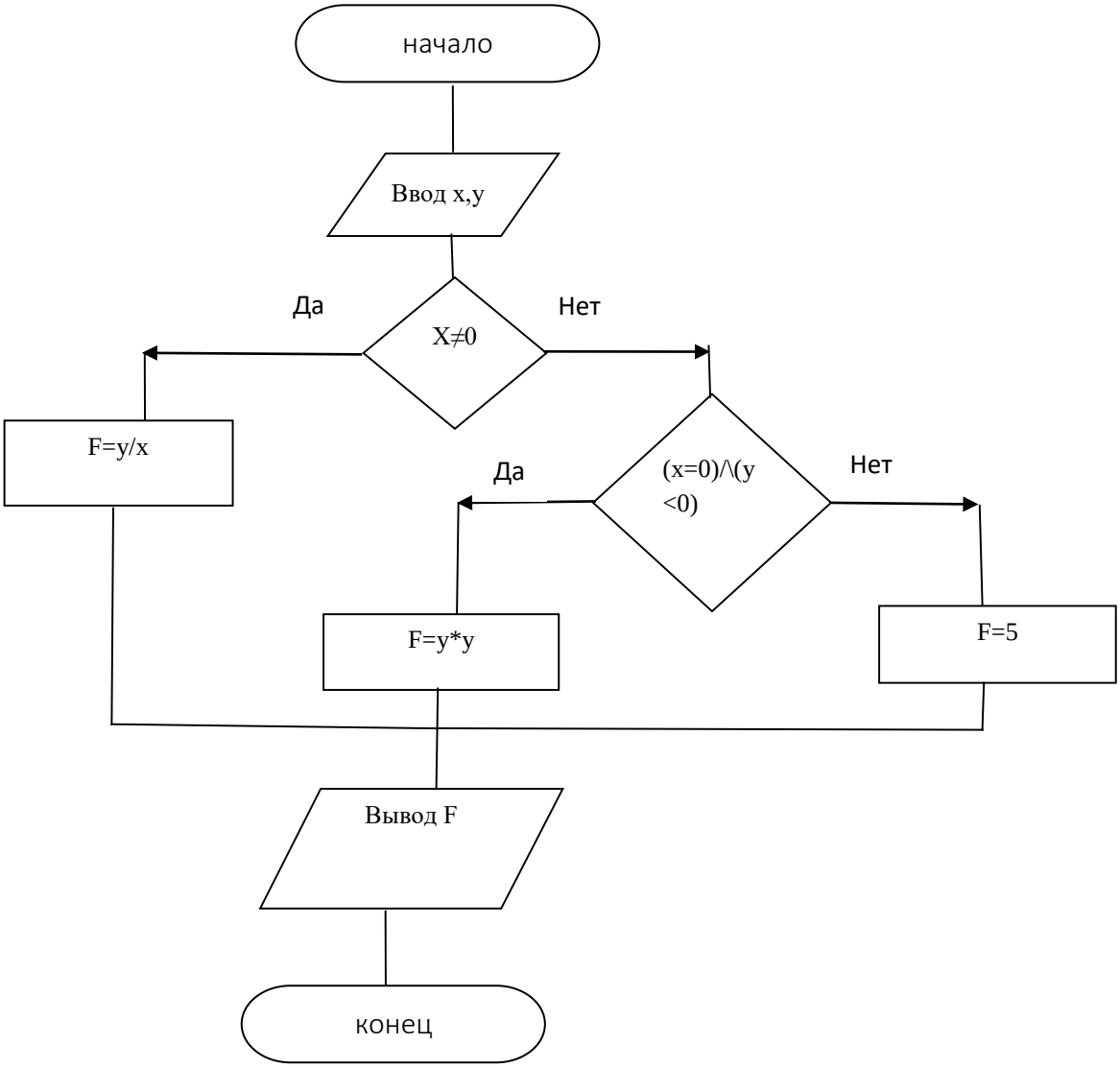
Результирующие данные:

Значение функции f , соответствующая переменная тоже будет типа double.

Таблица тестирования:

Входные данные	**Ожидаемый результат**	**Результат работы программы**
x=0; y=0	5	5.000000
x=3;y=6	2	2.000000
x=0 ; y=-2	5	5.000000

Схема программы:



Текст программы

```
#include <stdio.h>
int main(){
    double x,y,f;
    printf("enter x\n");
    scanf("%lf",&x);
    printf("enter y\n");
    scanf("%lf",&y);
    f=(x!=0)?(y/x):(x=0)&&(y<0)? (y*y):5;
    printf("f=%lf",f);
    return 0;
}
```

Задача 2. Вычислить значение функции $Q=\sqrt{\frac{\sin x+\cos^2 x}{2\ln(x^2+e^{-x})}}+\frac{1}{1+\frac{1}{2x}+|x|}$

Исходные данные:

Переменная x типа double.

Результирующие данные:

Значение функции Q типа double.

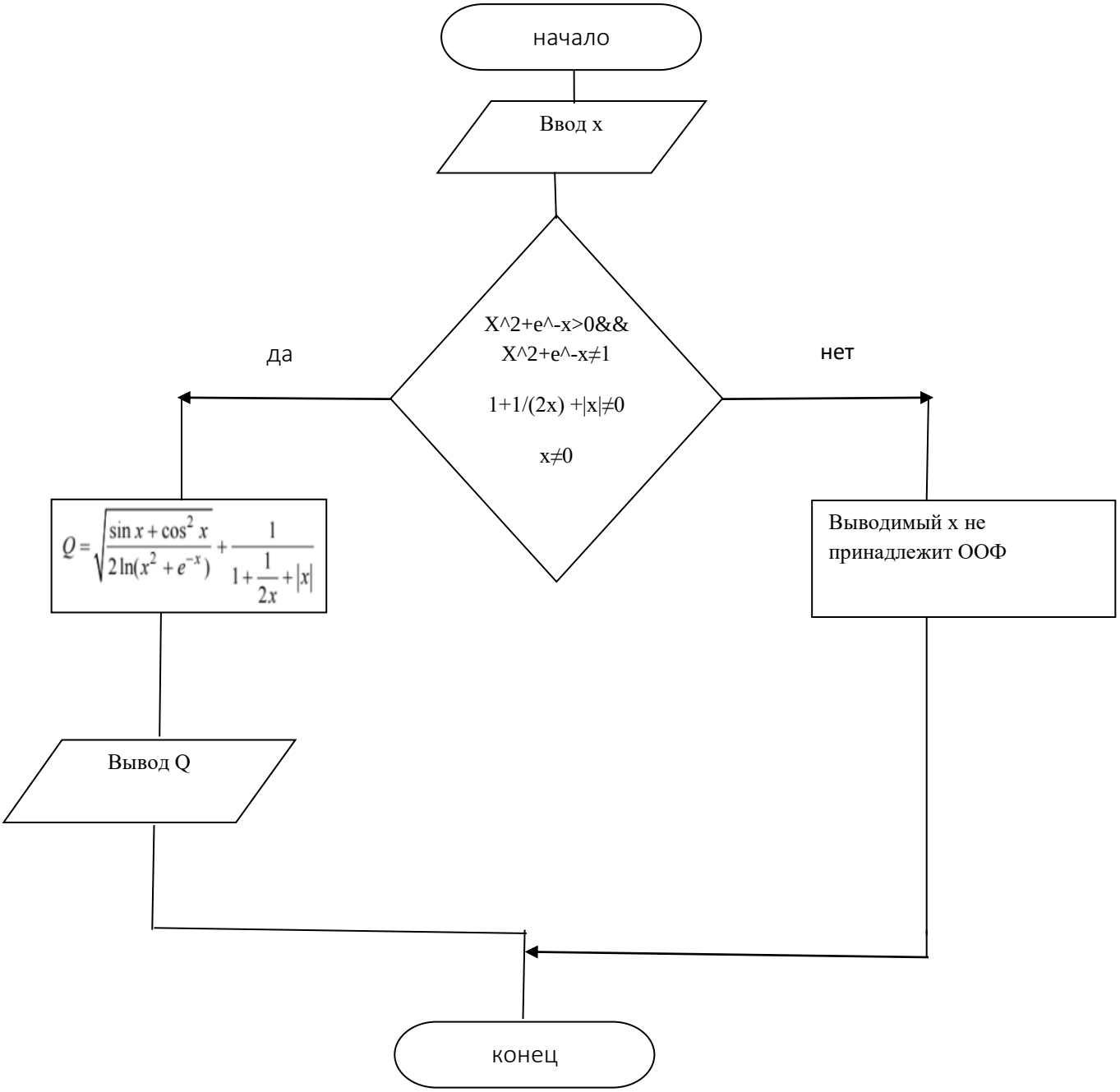
Предварительные вычисления:

Чтобы можно было вычислить значение функции, должны быть выполнены следующие условия: Знаменатель подкоренного выражения должен быть не равен 0,все подкоренное выражение >=0,подлогарифмическое выражение больше 0 и !=1,знаменатель второго слагаемого не равен 0,знаменатель выражения(1/2x) не равен 0.

Таблица тестирования:

Входные данные	Ожидаемый результат	Результат работы программы
x=1	1.745001	1.745001
x=0	"Введенные значения не входят в область определения функции"	"Введенные значения не входят в область определения функции"
x=2	0.925162	0.925162

Схема программы:



Текст программы

```
#include <stdio.h>
#include <math.h>
#include <locale.h>
int main(){
setlocale(LC_ALL, "Rus");
double x;
double q;
printf("x=");
scanf("%lf",&x);
if((fabs(x)>1E-15&&(pow(x,2)+exp(-
x))!=1)&&(1+1/(2*x)+fabs(x)!=0))/*знаменатель не равен 0, подкоренное выражение не равно 0 и модуль числа x не равен 0,поскольку сравнить вещественные числа с 0 некорректно ,то сравниваем x по модулю с приближенным к 0 значением.*/
{
q=(sqrt((sin(x)+cos(x)*cos(x))/(2*log((x*x)+exp(-
x)))))+1/(1+1/(2*x)+fabs(x));/*вычисление функции*/
printf("q=%lf",q);
}
else
printf("Введенные значения не входят в область определения функции");
return 0;
}
```

Задача 3. Точки с координатами (x1, y1) и (x2, y2) являются концами отрезка. Определить, пересекает ли этот отрезок график функции F(x) = x. Если пересекает, найти расстояние от каждой точки до данной прямой. Если не пересекает, определить, принадлежит ли этот отрезок прямой, параллельной данной, или прямой, ей перпендикулярной.

Исходные данные:

Переменные типа double : x1, y1, x2, y2(координаты точек)

Результирующие данные:

d1 и d2 типа double.(расстояния от точек до прямой)

.

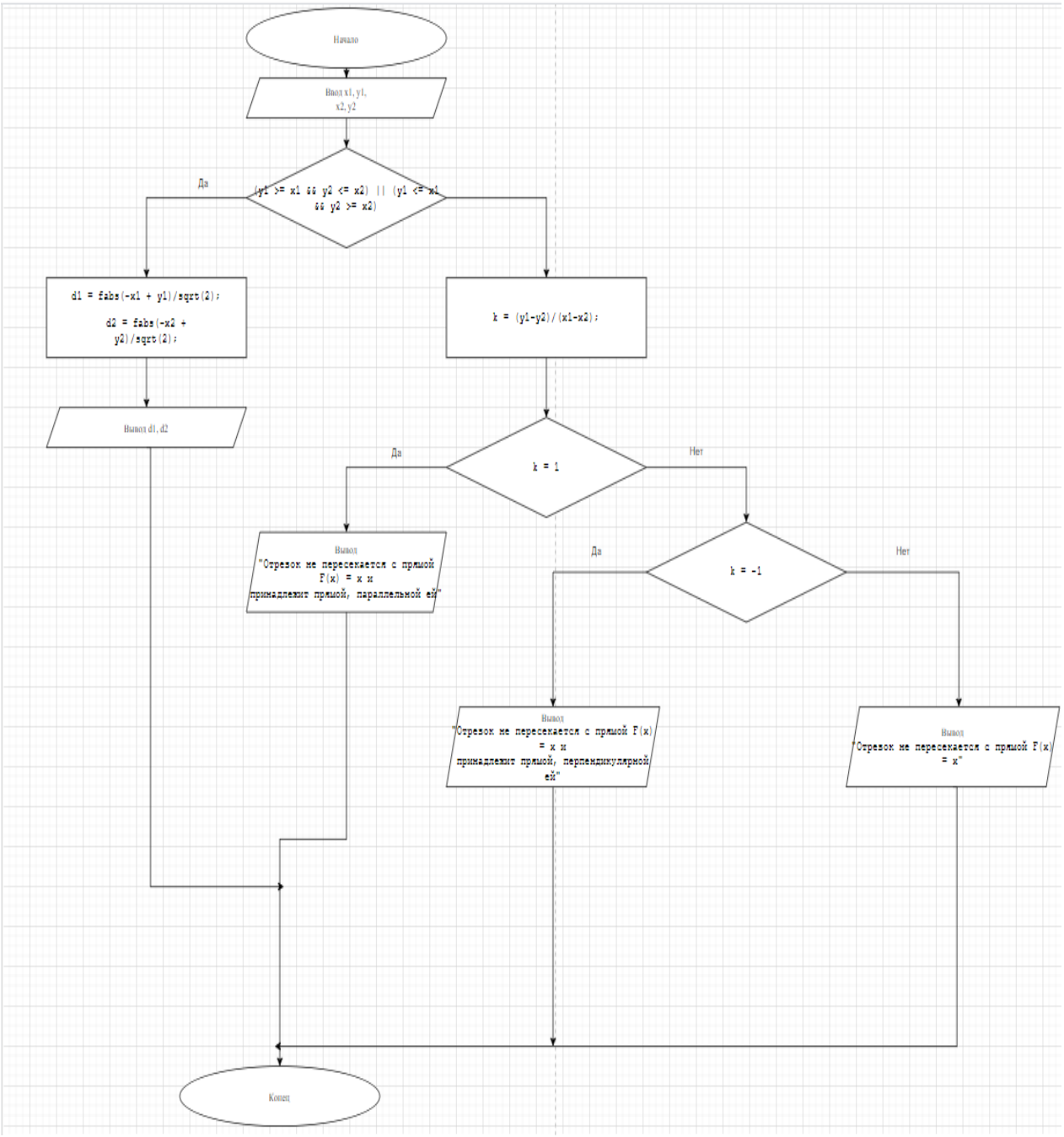
Вспомогательные переменные:

переменная k(касательная) типа double.

Таблица тестирования:

Входные данные	**Ожидаемый результат**	**Результат работы программы**
x1 = -1, y1 = -2, x2 = -2, y2 = 2	Программа выведет, что отрезок пересекает прямую F(x) = x, d1 = 0.7, d2 = 2.82	Отрезок пересекается с прямой F(x) = x Расстояние от точки (x1, y1) до прямой: d1 = 0.707107 Расстояние от точки (x2, y2) до прямой: d2 = 2.828427
x1 = 1, y1 = 1, x2 = 2, y2 =2	Программа выведет, что отрезок пересекает прямую F(x) = x, d1 =0, d2 = 0	Отрезок пересекается с прямой F(x) = x Расстояние от точки (x1, y1) до прямой: d1 = 0.000000 Расстояние от точки (x2, y2) до прямой: d2 = 0.000000
x1 = 1, y1 = 3, x2 = 4, y2 =2	Программа выведет, что отрезок пересекает прямую F(x) = x, d1 =1.41, d2 = 1.41	Отрезок пересекается с прямой F(x) = x Расстояние от точки (x1, y1) до прямой: d1 = 1.414214 Расстояние от точки (x2, y2) до прямой: d2 = 1.414214
x1 = 2.4, y1 = 2.6, x2 = 0, y2 =0.2	Программа выведет, что отрезок лежит на прямой параллельной прямой F(x) = x	Отрезок не пересекается с прямой F(x) = x и принадлежит прямой, параллельной ей
x1 = -2.3, y1 = 2.8, x2 = -2.9, y2 = 3.4	Программа выведет, что отрезок лежит на прямой перпендикулярной прямой F(x) = x	Отрезок не пересекается с прямой F(x) = x и принадлежит прямой, перпендикулярной ей
x1 = -2.6, y1 = 7.8, x2 = -1.9, y2 = 3.1	Программа выведет, что отрезок не пересекает прямую F(x) = x	Отрезок не пересекается с прямой F(x) = x

Схема программы:



Текст программы

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <locale.h>
int main(){
```

```
    setlocale(LC_ALL, "Rus");
double x1, x2, y1 , y2, d1, d2, k;/*объявление переменных*/
printf("Введите x1:");/*вывод надписи для пользователя*/
scanf("%lf", &x1);/*ввод значения x1*/
printf("Введите y1:"); /*вывод надписи для пользователя*/
scanf("%lf", &y1);/*ввод значения y1*/
printf("Введите x2:"); /*вывод надписи для пользователя*/
scanf("%lf", &x2);/*ввод значения x2*/
printf("Введите y2:"); /*вывод надписи для пользователя*/
scanf("%lf", &y2);/*ввод значения y2*/
if ((y1 >= x1 && y2 <= x2) || (y1 <= x1 && y2 >= x2)){/*проверка на
пересечение с прямой F(x)=x*/
d1 = fabs(-x1 + y1)/sqrt(2);
d2 = fabs(-x2 + y2)/sqrt(2);
printf("Отрезок пересекается с прямой F(x) = x\n");
printf("Расстояние от точки (x1, y1) до прямой: d1 = %lf\n",d1);
printf("Расстояние от точки (x2, y2) до прямой: d2 = %lf",d2);
}
else{
if(x1-x2==0)
printf("деление на 0 ");
else{
k = (y1-y2)/(x1-x2);/*k-касательная*/
if (k == 1){
printf("Отрезок не пересекается с прямой F(x) = x и принадлежит
прямой,параллельной ей\n");
}
else if(k == -1){
printf("Отрезок не пересекается с прямой F(x) = x и принадлежит
прямой,перпендикулярной ей\n");
}
else{
printf("Отрезок не пересекается с прямой  F(x) = x не принадлежит
прямой \n");
}
}}
return 0;
}
```

Задача 4. Дано натуральное число n. Вычислить F=1!+2!+...+n! Факториал числа n равен произведению чисел натурального ряда от 1 до n.

Исходные данные:

int x

Результирующие данные:

Int f(факториал x)

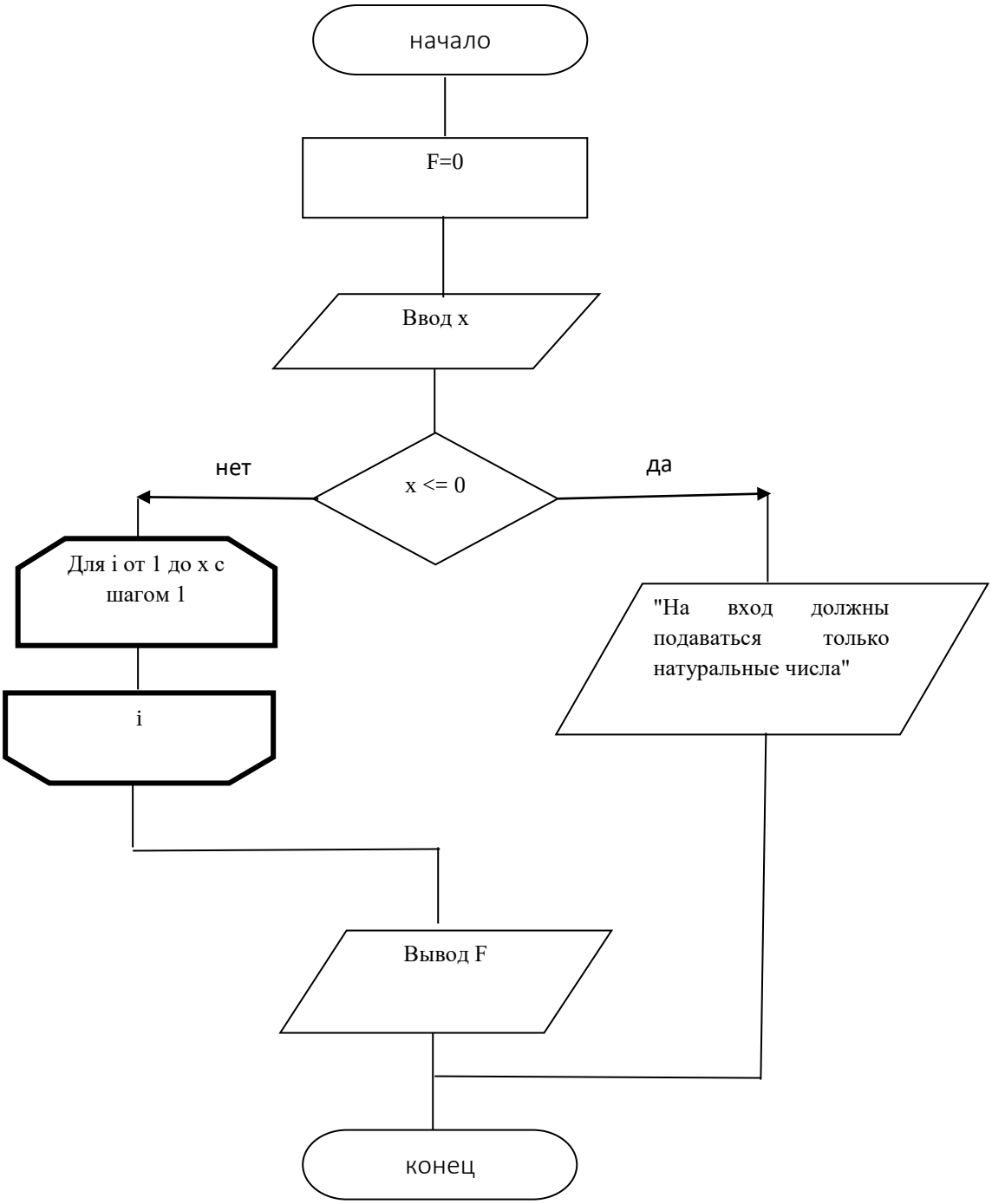
Вспомогательные переменные:

Int i (счетчик)

Таблица тестирования:

Входные данные	**Ожидаемый результат**	**Результат работы программы**
3	*6*	*6*
0	*На вход должны подаваться только натуральные числа*	*На вход должны подаваться только натуральные числа*
-3	*На вход должны подаваться только натуральные числа*	*На вход должны подаваться только натуральные числа*

Схема программы:



Текст программы:

```
#include <math.h>
int main(){
   int x, i, f=1;
   printf("x = ");
   scanf("%d", &x);
   if (x <= 0) /*проверка числа на натуральность*/
   {
      printf("На вход должны подаваться только натуральные числа");
      return 0;
   }
   for(i=1; i<=x; i++)
      f *= i; /*цикл for  для подсчета факториала */
   printf("факториал = %d", f);
   return 0;
}
```

Задача 5

Вывести на экран значение первого отрицательного члена последовательности cos(ctg(n)), n = 1,2,3,.... Отрицательные члены в

этой последовательности обязательно есть.

Исходные данные:

Переменная n типа float

Результирующие данные:

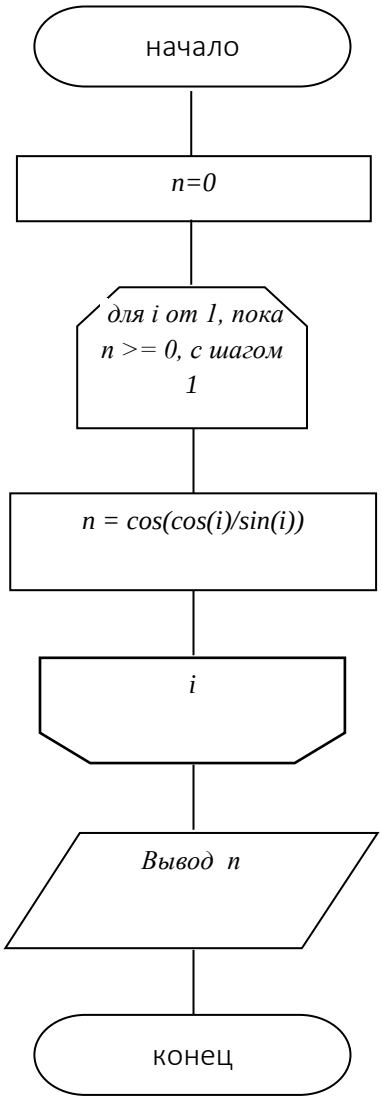
Переменная n типа float

Вспомогательные переменные:

Переменная i, типа int

Результат:

Схема программы:



Текст программы:

```
#include <stdio.h>
#include <math.h>
int main() {
```

```
float  n = 0;
int i=1;
while(n >= 0)
    n = cos(cos(i)/sin(i++)); /*перебор аргумента котангенса пока n>=0*/
printf("%lf", n);
return 0;
}
```

Задача 6.

Предприниматель, начав дело, взял кредит размером k рублей

под p процентов годовых и вложил его в свое дело. По прогнозам, его дело должно давать прибыль r рублей в год. Сможет ли он накопить сумму,

достаточную для погашения кредита, и если да, то через сколько лет?

Исходные данные:

Переменные типа double (k-кредит,p-процент по кредиту,r-прибыль)

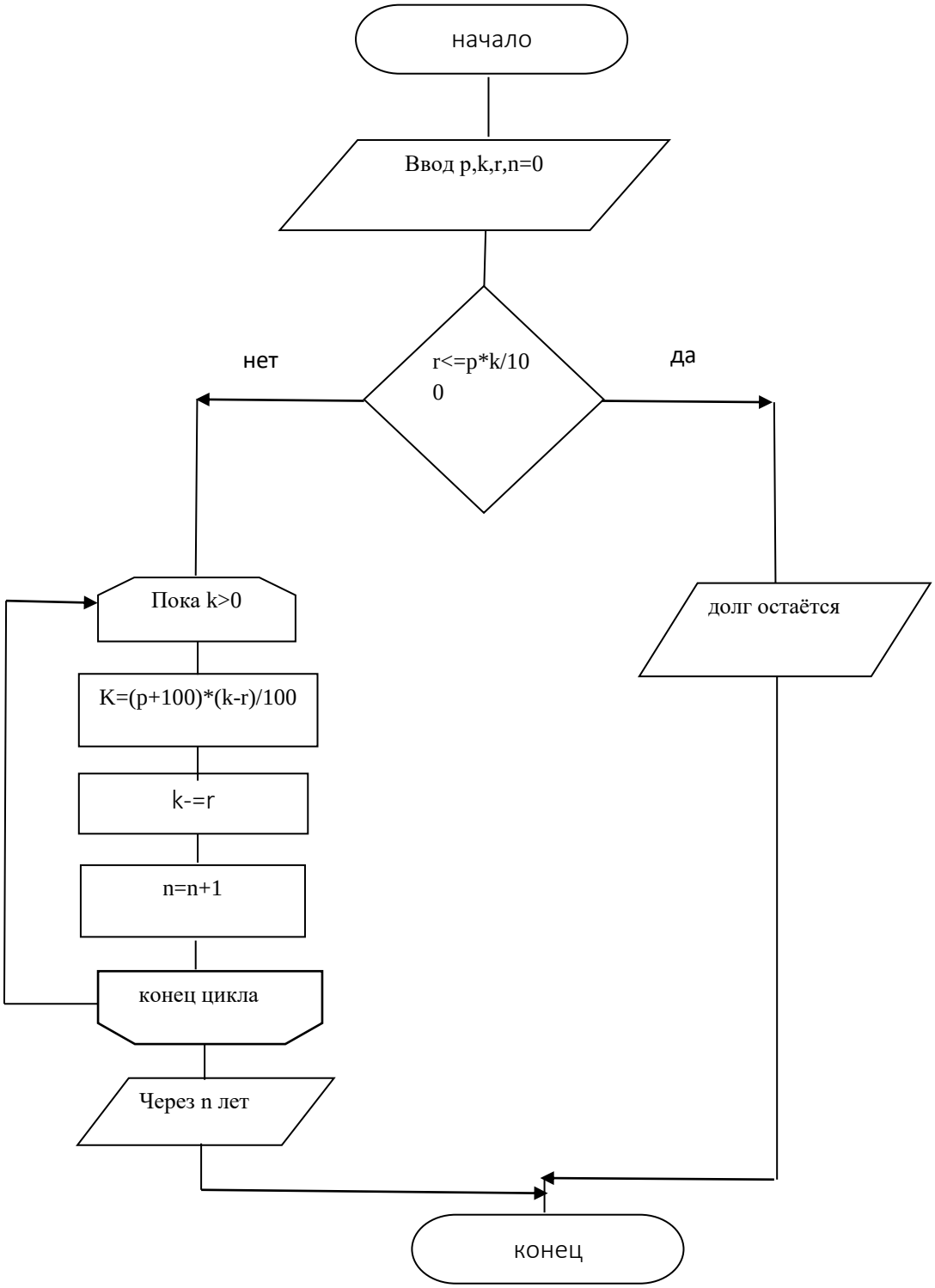
Результирующие данные:

Переменная n типа int

Таблица тестирования:

Входные данные	Ожидаемый результат	Результат работы программы
K=1000000;p=20%;r=300000	Кредит погашен через 7 лет	Кредит погашен через 7 лет
K=1000000;p=20%;r=500000	Кредит погашен через 3 лет	Кредит погашен через 3 лет
k=100000;p=10%;r=30000	Кредит погашен через 5 лет	Кредит погашен через 5 лет

Схема программы:



Текст программыж;

```
#include <stdio.h>
#include <stdlib.h>

#include <stdio.h>
#include <locale.h>
int main(){
   setlocale(LC_ALL, "Rus");
   double k,p,r;
   int n=0;
   printf("k=");
   scanf("%lf",&k);
   printf("p=");
   scanf("%lf",&p);
   printf("r=");
   scanf("%lf",&r);
   if (r<=p*k/100)/*k=кредит ; p- % по кредиту;r-прибыль.Если прибыль меньше начисляемых проценов то долг остается.*/
      printf("долг остаётся \n");
   else
   {
      do /*пока сумма кредита больше чем 0  происходит ее погашение*/
      {
         k=(p+100)*k/100;
         k-=r;
         n++;
      }
      while (k>0);
      printf("Кредит погашен через %d лет",n);
   }
   system("pause");
   return 0;
}
```

Задача 7. Вычислить значение суммы бесконечного ряда с заданной точностью ε=10^{-5} $S = 1 - \frac{x^2}{3!} + \frac{x^4}{5!} - \frac{x^6}{7!} + \cdots$ и значение функции (для проверки) $f = \frac{\sin x}{x}$

Использовать рекуррентные зависимости для вычисления значений слагаемых.

Исходные данные:

Аргумент функции. Тип аргумента в задании не указан, поэтому переменная x будет типа double.

Результирующие данные:

Значение суммы s тоже будет типа double.

Вспомогательные переменные:

i – индекс слагаемого – целое число типа int, *a – значение текущего слагаемого – вещественное число типа double.*

Предварительные вычисления:

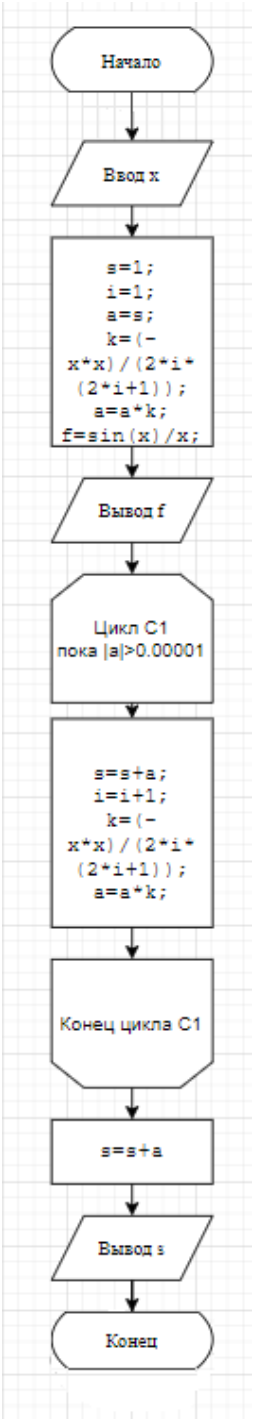
i-ое слагаемое $a_i = \frac{(-1)^i x^{2i}}{(2i+1)!}$,

предшествующее слагаемое $a_{i-1} = \frac{(-1)^{i-1} x^{2(i-1)}}{(2(i-1)+1)!} = \frac{(-1)^{i-1} x^{2i-2)}}{(2i-1)!}$

коэффициент пропорциональности $k_i = \frac{a_i}{a_{i-1}} = \frac{\frac{(-1)^i x^{2i}}{(2i+1)!}}{\frac{(-1)^{i-1} x^{2i-2)}}{(2i-1)!}} = \frac{-x^2}{4i^2+2i}$

индекс первого слагаемого в рекуррентной последовательности i=0, первое слагаемое $a_0 = \frac{(-1)^0 x^{2*0}}{(2*0+1)!} = \frac{1*1}{1!} = 1$

Схема программы:



Текст программы

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <math.h>
int main(){
    double x, s, k, f, a;
    int i, n;
    do{
        printf("Введите x:");
        n = scanf("%lf", &x);
    } while(n == 0 || x < -M_PI || x > M_PI);*/ввод числа x находящегося в диапазоне от -p до p(пока x не будет соответствовать условиям задачи,программа заставляет пользователя заново его вводить)*/
    s = 1;/*сумма(первый член последовательности 1)*/
    i = 1; /*счетчик*/
    a = s;
    k = (-x * x) / (2 * i * (2 * i + 1));
    a *= k;/*вычисление члена последовательности*/
    f = sin(x) / x;
    printf("f = %lf\n", f);
    while(fabs(a) >= 0.00001){ /*цикл подсчета суммы бесконечного ряда,с точностью 10^-5,с использованием рекуррентных зависимостей*/
        s += a;/*прибавление члена последовательности к сумме*/
        i++;/*прибавление счетчика*/
        k = (-x * x) / (2 * i * (2 * i + 1));
        a *= k;
    }
    s += a;
    printf("S = %lf", s);
    return 0;
}
```

Результаты тестирования

Исходные данные	**Ожидаемый результат**	**Результат работы программы**
x = 10	Программа попросит пользователя вновь ввести x	Программа просит пользователя ввести x заново
x = 1	0.841471	0.841471
x = 0	1	1.000000
x = -0.25	0.989616	0.989616

<u>Задача 8</u>. Вычислить значение суммы бесконечного ряда $S = 1 - \frac{1}{4}x + \frac{1*5}{4*8}x^2 - \frac{1*5*9}{4*8*12}x^3 + \frac{1*5*9*13}{4*8*12*16}x^4 - \ldots$ с точностью до члена ряда, по модулю меньшего $\varepsilon = 10^{-4}$, и значение функции (для проверки)$f = (1+x)^{-1/4}$,

учесть, что $x^2 < 1$. Использовать рекуррентные зависимости для вычисления значений слагаемых по частям.

Исходные данные:

Переменна x типа double.

Результирующие данные:

Значение суммы s и значение функции f тоже будут типа double.

Вспомогательные переменные:

i – индекс слагаемого – целое число типа int, a – значение текущего слагаемого – вещественное число типа double, k – коэффициент пропорциональности – вещественное число типа double.

Предварительные вычисления:

-е слагаемое $a_3 = -\frac{1*5*9}{4*8*12}x^3$,

предшествующее слагаемое $a_2 = \frac{1*5}{4*8}x^2$

предшествующее слагаемое $a_1 = -\frac{1}{4}x$

находим коэффициенты k_2 и k_3

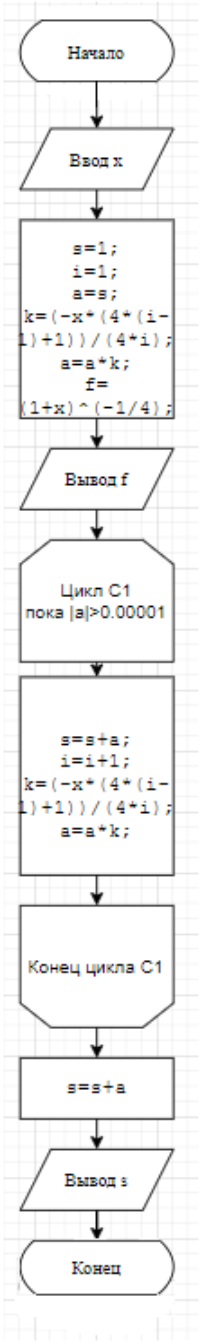
$$k_2 = \frac{a_2}{a_1} = \frac{\frac{1*5}{4*8}x^2}{-\frac{1}{4}x} = -\frac{5x}{8}$$

$$k_3 = \frac{a_3}{a_2} = \frac{-\frac{1*5*9}{4*8*12}x^3}{\frac{1*5}{4*8}x^2} = -\frac{9x}{12}$$

Внимательно изучив данную последовательность, мы приходим к выводу, что коэффициент пропорциональности равен $k_i = \frac{-x(4(i-1)+1)}{4i}$

индекс первого слагаемого в рекуррентной последовательности i=0, первое слагаемое $a_0 = 1$

Схема программы:



Текст программы

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
int main(){
    double x, s, k, f, a;
    int i, n;
```

```
    do{
        printf("Введите x:");
        n = scanf("%lf", &x);
    } while(n == 0 || x * x >= 1);/*ввод числа X,квадрат которого больше или равен 1(пока x не  будет соответствовать условиям задачи,программа заставляет пользователя заново его вводить)*/
    s = 1;/*сумма(первый член последовательности 1)*/
    i = 1;/*счетчик*/
    a = s;
    k = (-x * (4 * (i - 1) + 1)) / (4 * i);
    a *= k; /*вычисление члена последовательности*/
    f = pow((1 + x), -0.25);
    printf("f = %lf\n", f);
    while(fabs(a)  >=  0.0001){  /*цикл подсчета суммы бесконечного ряда,с точностью 10^-5,с использованием рекуррентных зависимостей*/
        s += a; ;/*прибавление члена последовательности к сумме*/
        i++;/*прибавление счетчика*/
        k = (-x * (4 * (i - 1) + 1)) / (4 * i);
        a *= k;
    }
    s += a;
    printf("S = %lf", s);
    return 0;
}
```

Результаты тестирования:

Исходные данные	**Ожидаемый результат**	**Результат работы программы**
x = 10	Программа попросит пользователя вновь ввести x	Программа просит пользователя ввести x заново
x = -9	Программа попросит пользователя вновь ввести x	Программа просит пользователя ввести x заново
x = 0	1	1
x = 0.25	0.945742	0.945727
x = -0.14	1.038426	1.038421

Задача 9

Найти все натуральные числа, не превосходящие заданного n,

десятичная запись которых есть строго возрастающая или строго убывающая последовательность цифр.

Исходные данные:

Переменная n типа int

Результирующие данные:

Переменная k типа int

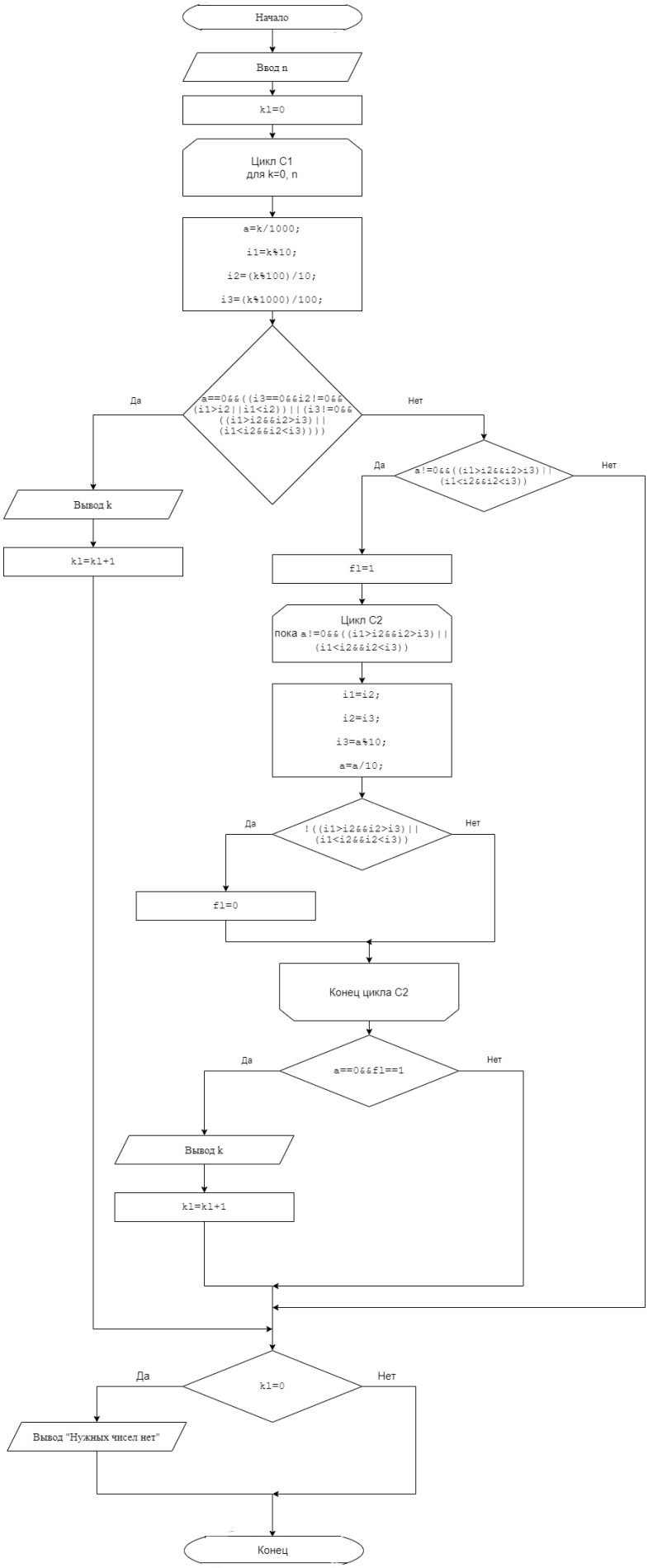
Вспомогательные переменные:

i1, i2, i3 – разряды числа типа int, a – переменная, в которую будет записываться число k для операций над ним типа int, kl – количество чисел, удовлетворяющих условию, типа int, fl – вспомогательная переменная для исключения ошибки в последних разрядах типа int.

Таблица тестирования:

Входные данные	**Ожидаемый результат**	**Результат работы программы**
n = 6	Программа попросит пользователя вновь ввести x	Программа просит пользователя ввести x заново
n = -15	Программа попросит пользователя вновь ввести x	Программа просит пользователя ввести x заново
n = 0	Программа выведет, что нужных чисел нет	Нужных чисел нет
n=100	10, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98	10, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98

Схема программы:


Начало
Ввод n
k1=0
Цикл C1
для k=0, n
a=k/1000;
i1=k%10;
i2=(k%100)/10;
i3=(k%1000)/100;
a==0&&((i3==0&&i2!=0&&(i1>i2||i1<i2))||(i3!=0&&((i1>i2&&i2>i3)||(i1<i2&&i2<i3))))
Да
Нет
Вывод k
k1=k1+1
a!=0&&((i1>i2&&i2>i3)||(i1<i2&&i2<i3))
Да
Нет
f1=1
Цикл C2
пока a!=0&&((i1>i2&&i2>i3)||(i1<i2&&i2<i3))
i1=i2;
i2=i3;
i3=a%10;
a=a/10;
!((i1>i2&&i2>i3)||(i1<i2&&i2<i3))
Да
Нет
f1=0
Конец цикла C2
a==0&&f1==1
Да
Нет
Вывод k
k1=k1+1
k1=0
Да
Нет
Вывод "Нужных чисел нет"
Конец

Текст программы:

```
#include <stdio.h>

#include <stdlib.h>

#include <math.h>

int main(){

    int n, s, k, a, i1, i2, i3, fl, kl;

        printf("Введите n:");

        scanf("%d", &n);

        if( n >= 0){/*проверка числа на неотрицательность*/

        kl = 0;

    for(k = 0; k <= n; k++){/*перебор неотрицательных значений до числа n*/

        a = k / 1000;

        i1 = k % 10;/*присваивание разрядов числа переменным*/

        i2 = (k % 100) / 10;

        i3 = (k % 1000) / 100;

        if( a == 0 && ((i3 == 0 && i2 != 0 && (i1 > i2 || i1 < i2)) || (i3 !=
0 && ((i1 > i2 && i2 > i3) || (i1 < i2 && i2 < i3))))){/*Если число
двухзначное и его цифры убывают или возрастают или число трёхзначное и его
цифры возрастают или убывают, то выводим это число и увеличиваем счётчик*/

            printf("%d ", k);

            kl += 1;

        }

        else if((a != 0 && i2 > i3) || (i1 < i2 && i2 < i3)){/*проверка
разрядов на возрастание*/

            fl = 1;

            while(a != 0 && ((i1 > i2 && i2 > i3) || (i1 < i2 && i2 <
i3))){/*добавление дополнительно младшего разряда*/

                i1 = i2;

                i2 = i3;

                i3 = a % 10;

                a /= 10;

                if (!((i1 > i2 && i2 > i3) || (i1 < i2 && i2 <
i3))){/*проверка на переполнение разрядов*/
```

```
                fl = 0;
            }
        }
        if (a == 0 && fl == 1){/*увеличение счетчика на 1*/
            printf("%d ", k);
            kl += 1;
        }
    }
}
if(kl == 0)
    printf("Нужных чисел нет");
    }
return 0;
}
```